

Project Report: Professional Placement Portal – MAD II

Student Name: Anas Kizhakkanaly Abdul Assis

Student ID: 23f1001741

Course: Modern Application Development II

Project Details

Problem Statement:

The goal was to develop a multi-user web application to manage institutional placements. The system requires distinct roles: an Admin with full control over companies and students, and Users (Students and Companies) who can register, log in, and manage placement activities.

Approach:

The problem was approached using a decoupled architecture consisting of a Flask-based Backend for logic and a Vue.js-based Frontend for the user interface. Application logic was structured into separate files (admin, company, and student) to maintain organized code. I integrated Celery with Redis to handle background tasks.

Technologies and Frameworks Used

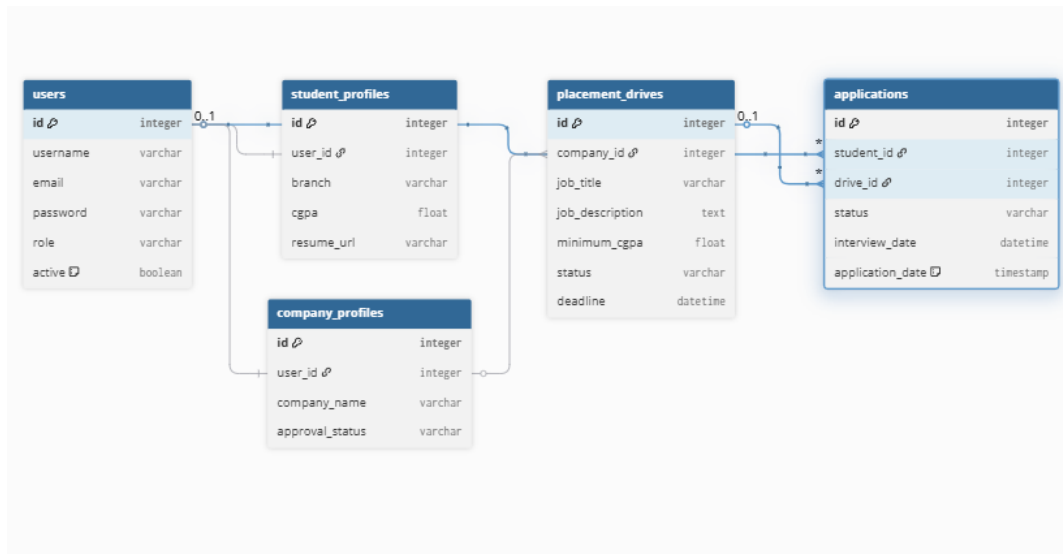
Technology / Library	Purpose
Flask	Core backend web framework
Vue.js 3	Frontend framework for a reactive user interface
SQLAlchemy	Object Relational Mapper for database management
Flask-JWT-Extended	Secure Token-based Authentication and Role-Based Access Control
SQLite	Lightweight local database for storing application data
Celery & Redis	Tools for managing background jobs and scheduled reports
Chart.js	Data visualization for the admin dashboard
Bootstrap 5	Frontend styling and responsive design
MailHog	Local SMTP server for testing automated email notifications

Database Schema / ER Diagram

Key Entities:

- **User:** Stores credentials and role-specific flags.
- **StudentProfile:** Stores student-specific data like CGPA and branch.
- **CompanyProfile:** Manages company details and approval status.
- **PlacementDrive:** Represents specific job openings and eligibility criteria.

- **Application:** Tracks the status of student applications to specific drives.



API Resource Endpoints

Authentication & Registration:

Endpoint	Method	Description
/api/auth/register	POST	Handles self-registration for new Student and Company accounts.
/api/auth/login	POST	Authenticates credentials and returns a secure JWT token and user role.

Admin Management:

Endpoint	Method	Description
/api/admin/dashboard	GET	Returns global statistics (Total Students, Companies, and Drives).
/api/admin/users	GET	Searchable list of all system users with activation status.
/api/admin/users/<id>/toggle-status	POST	Used to blacklist or reactivate specific students or companies.
/api/admin/companies/pending	GET	Fetches company profiles awaiting registration approval.
/api/admin/companies/<id>/status	POST	Admin action to Approve or Reject a company's registration.
/api/admin/drives/pending	GET	Fetches placement drives awaiting verification.
/api/admin/drives/<id>/status	POST	Admin action to Approve or Reject a specific placement drive.
/api/admin/interviews	GET	Global master schedule of all upcoming interviews across the portal.

Company:

Endpoint	Method	Description
/api/company/stats	GET	Fetches specific metrics for the logged-in company's dashboard.
/api/company/drives	POST	Creates a new placement drive (requires Admin approval).
/api/company/drives	GET	Lists all drives created by the specific company.
/api/company/drives/<id>/applications	GET	Retrieves all students who applied to a specific drive.
/api/company/applications/<id>/status	PUT	Updates a student's status (e.g., Shortlisted, Selected, Rejected).
/api/company/applications/<id>/schedule	POST	Sets the interview date and time for a shortlisted student.
/api/company/drives/<id>/export	POST	Triggers an asynchronous Celery task to email a CSV of applicants.

Student:

Endpoint	Method	Description
/api/company/stats	GET	Fetches specific metrics for the logged-in company's dashboard.
/api/company/drives	POST	Creates a new placement drive (requires Admin approval).
/api/company/drives	GET	Lists all drives created by the specific company.
/api/company/drives/<id>/applications	GET	Retrieves all students who applied to a specific drive.
/api/company/applications/<id>/status	PUT	Updates a student's status (e.g., Shortlisted, Selected, Rejected).
/api/company/applications/<id>/schedule	POST	Sets the interview date and time for a shortlisted student.
/api/company/drives/<id>/export	OST	Triggers an asynchronous Celery task to email a CSV of applicants.

Architecture and Features

The application is built using a decoupled architecture that separates the frontend user interface from the backend logic, ensuring a smooth and responsive experience for all user roles.

- **Modular API-First Design:** Application logic and routing are managed within dedicated Blueprint-based modules (Admin, Company, and Student). This separation of concerns simplifies maintenance and allows each role-based service to scale independently.
- **Scheduled Batch Jobs:** The system utilizes Celery Beat to manage time-sensitive automated tasks. It sends daily email reminders to students regarding upcoming application deadlines and generates a comprehensive placement performance report for the admin on the first day of every month.
- **Dynamic Eligibility Validation:** A robust validation engine is implemented to enforce recruitment standards. Students are automatically restricted from applying to specific drives if their profile CGPA does not meet the company's minimum requirement, with real-time feedback provided via the UI.

- **Asynchronous Background Processing:** To maintain high performance and prevent UI freezing, data-intensive tasks—such as exporting student application history or company applicant lists as CSV files—are handled asynchronously by Celery workers.
 - **Stateless Authentication:** The system uses JWT (JSON Web Tokens) for role-based access control, allowing for a secure and stateless communication layer between the Vue.js frontend and the Flask API.
 - **Data Integrity & Archiving:** Following a robust approach for real-world applications, a "soft-delete" system is implemented using an active flag. This ensures that historical placement data remains available for reports even after a user or company is deactivated.
-

Video Presentation

- **Drive Link:** [Placement Portal Application - Demo](#)
-

AI/LLM Declaration

I utilized LLM to assist in the development of this project. The overall usage is estimated at **20–25%**.

- **Structural Guidance:** AI was used to assist in defining the initial structures and the basic layout of the Vue.js components.
- **Technical Troubleshooting:** I consulted AI to help correct implementation errors and debug configuration issues related to the Celery and Redis environment.
- **Interface Refinement:** AI provided suggestions for CSS properties to ensure a responsive design and helped generate sample code for the dashboard charts.
- **Data Testing:** AI was used to generate synthetic data to populate the database during the testing phase.

Statement of Authenticity: While AI provided support for coding patterns and troubleshooting, the core logic, eligibility validation rules, and final system integration were performed manually by me.
